

임장혁

문제를 기능이 아닌 시스템 관점에서 해결하는 백엔드 개발자입니다.

단순한 기능 구현을 넘어 "왜 이런 문제가 발생하는가?", "왜 이런 구조가 필요한가?"를 고민하며, 실험과 수치 기반으로 시스템을 개선하는 개발을 지향합니다.

Backend Developer Java · Spring Boot · TypeScript · NestJS · LLM github.com/imjanghyeok
dlaljh.tistory.com qkdrhkgn23@naver.com

문제를 바라보는 방식

기능이 동작하는지보다, 실패했을 때 사용자가 무엇을 잃는지를 먼저 봅니다.

- 사용자에게는 지연, 반복, 누락이 모두 신뢰 하락으로 보일 수 있다고 가정합니다.
- 런타임 상태와 영속 데이터를 분리하고, 재시작·재접속·timeout 이후 복구 가능성을 확인합니다.
- 선택한 구조가 감수한 비용까지 기록해 다음 운영 판단의 기준으로 남깁니다.

주요 경험 축

- **State Modeling** Redis unread, TTL session, Scheduler 상태 전이
- **Flow Control** Kafka ordering, WebSocket fan-out, LLM 종료 정책
- **Responsibility** 도메인 객체, Consumer, 서버 정책의 변경 이유 분리
- **Validation** 실험, 성능 수치, CI/E2E 기반 회귀 검증

프로젝트에서 반복적으로 다룬 기술 영역

Backend Java · Spring Boot · NestJS · Node.js · TypeScript

Message / Cache Kafka · Redis · WebSocket · STOMP

Data MySQL · PostgreSQL · MongoDB · JPA · TypeORM

Infra Docker · Docker Compose · GitHub Actions · AWS · NCP · Nginx

01. Synapse

AI 면접 시뮬레이션 서비스

2025.12 - 2026.02 · Backend · NestJS · TypeORM · MySQL · Docker · GitHub Actions

PRIMARY ENGINEERING DECISION

LLM은 candidate generator로 제한하고, 종료·중복·재시도는 deterministic control layer가 enforce한다.

왜 중요했나

프롬프트가 좋아도 LLM 출력은 매번 흔들릴 수 있습니다. 면접 서비스에서 이 흔들림은 질문 품질 문제가 아니라 “면접이 진행되고 있다”는 사용자 경험의 신뢰 문제였습니다.

1. 프로젝트 개요

사용자의 포트폴리오를 기반으로 AI 면접관이 개인화된 질문과 피드백을 제공하는 서비스입니다.

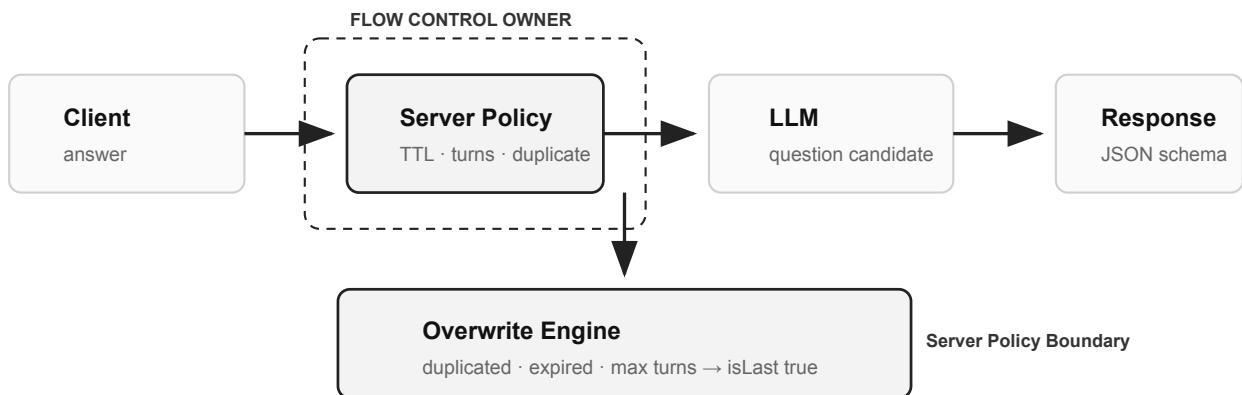
내 역할: 면접 흐름 제어, 질문 생성 상태 관리, TTL cleanup, 삭제 정책, CI/E2E, Docker 배포 최적화.

핵심 문제

사용자는 질문이 반복되거나 종료가 어색하면 “AI가 내 답변을 이해하지 못한다”고 느낍니다. 그래서 LLM은 질문 후보만 만들고, 면접 흐름의 최종 판단은 서버 정책이 말도록 분리했습니다.

15.1% 잔존 중복률 첫 질문 100회 생성 기준	5배 삭제 성능 개선 cascade delegation으로 round trip 감소	81% 메모리 절감 multi-stage build + dependency pruning	4.14 / 5 사용자 피드백 최종 발표 14명 응답 기준
---	---	--	---

Measurement note: 질문 중복률은 동일 포트폴리오 입력으로 첫 질문을 100회 생성해 계산한 개선 후 잔존 중복률입니다. 삭제 성능과 메모리 수치는 팀 로컬/배포 테스트 환경의 전후 비교이며, 운영 트래픽 수치가 아니라 구조 변경 효과를 확인한 지표입니다.



LLM은 질문 후보를 생성하고, 서버 정책이 중복·종료·재시도 흐름을 최종 결정합니다.

문제 1. LLM 비결정성으로 면접 흐름이 깨지는 문제

사용자에게 반복 질문과 어색한 종료는 “AI가 나를 이해하지 못한다”는 신호입니다. LLM 출력은 후보로만 두고, 중복/종료 판단은 서버 정책으로 제한했습니다.

실제로 부딪힌 문제

테스트 중 짧은 답변이나 애매한 답변이 들어오면 꼬리질문 품질이 급격히 낮아지고, 종료 신호가 늦어져 비슷한 질문이 반복될 수 있었습니다. 그래서 LLM의 `isLast`를 그대로 믿지 않고 서버가 최종 종료 조건을 다시 계산했습니다.

PROBLEM	DECISION
prompt engineering만으로는 중복, 종료, 재시도 실패를 안정적으로 통제하기 어려웠습니다.	LLM은 후보 생성자, 서버는 deterministic policy layer로 분리했습니다.

CORE IMPLEMENTATION

Overwrite Engine

Termination Mode

JSON Schema

결론: 면접 흐름이 LLM 출력이 아니라 서버 정책 안에서 동작하도록 바뀌었습니다.

DEEP DIVE

Session & Cleanup

문제 2. 면접 중 임시 상태를 어디에 저장할 것인가

면접 중 상태가 남으면 메모리 누수와 잘못된 중복 판단으로 이어집니다. 그래서 면접 진행 상태를 TTL이 있는 런타임 상태로 정의했습니다.

예상과 달랐던 점

처음에는 세션별 timer가 가장 단순해 보였지만, 면접 세션이 늘면 timer leak과 cleanup timing을 추적하기 어려워질 수 있었습니다. 만료 시각 기준으로 한 곳에서 정리하는 구조가 필요했습니다.

PROBLEM	DECISION
질문 이력, 방문 주제, 던 수는 면접이 끝나면 즉시 사라져야 하는 상태였습니다.	상태 보관은 `KeySetStore`, 만료 관리는 `MinHeapScheduler`가 소유하게 했습니다.

CORE IMPLEMENTATION

KeySetStore

MinHeapScheduler

TTL cleanup

결론: 영속 데이터와 흐름 제어용 임시 상태의 저장 계층을 분리했습니다.

문제 3. 연관 엔티티 삭제 시 정합성과 성능을 동시에 보장하는 방법

연관 삭제 정합성은 DB 제약으로 보장하고, 서비스 계층은 삭제 요청 검증과 응답 책임에 집중시켰습니다.

PROBLEM

문서 삭제 시 자식 데이터 누락과 대량 삭제 성능 저하가 함께 발생할 수 있었습니다.

DECISION

삭제 정합성은 DB에 위임하고, 애플리케이션은 정책 검증에 집중했습니다.

CORE IMPLEMENTATION

onDelete: CASCADE

delete()

Document domain relation

결론: 연관 데이터 삭제 성능을 약 5배 개선하고 고아 데이터 위험을 줄였습니다.

OPERATIONAL DECISION

Operation & Limits

운영 보강

Migration: 운영 환경에서 `synchronize` 의존을 제거하고, 스키마 변경을 TypeORM Migration 파일과 배포 절차 안에서 검토하도록 전환했습니다.

CI/E2E: AI API가 포함된 인터뷰 흐름을 수동 확인에만 맡기지 않기 위해 Lint, Build, Unit, E2E 단계를 분리한 품질 게이트를 구성했습니다.

Docker: 1GB RAM 서버에서 배포 실패 가능성을 줄이기 위해 멀티 스테이지 빌드와 `.dockerignore`를 적용했고, 이미지 크기를 1.2GB에서 250MB로 줄였습니다.

Measurement: 삭제 성능은 동일 연관 데이터 기준 ORM `remove()`와 DB cascade `delete()`를 비교했고, 메모리는 동일 서버에서 컨테이너 실행 후 RSS 기준으로 비교했습니다.

Team Process: 설계 이슈와 PR 리뷰에 실패 조건을 같이 남겨 “왜 이 구조를 택했는지”를 팀원이 따라갈 수 있게 했습니다.

OPERATIONAL DECISION

단일 인스턴스 검증 단계에서는 인메모리 TTL로 충분했습니다. 운영 전환 기준은 동시 면접 세션 증가, 인스턴스 2대 이상 scale-out, 세션 유실 허용 불가 시점입니다. 이때 Redis session store로 이전하고, LLM timeout·retry 횟수·fallback termination·duplicate guardrail 실패 로그를 모니터링 대상으로 뒤야 합니다.

실제 구현

인메모리 TTL 세션 저장소, MinHeapScheduler, Overwrite Engine, TypeORM Cascade 삭제, CI/E2E, Docker 경량화

이후 고려한 방향

Redis session store, LLM timeout/fallback, output evaluator, duplicate guardrail, 검색 기반 컨텍스트 주입

회고

확률적 출력을 서비스에 넣을 때 핵심은 더 긴 프롬프트가 아니라, 실패했을 때 서버가 어디서 멈추고 복구할지 정하는 일이었습니다.

02. SmileTogether

MSA 기반 협업 메신저

2025.01 - 2025.03 · Backend Lead · Spring Boot · Kafka · MongoDB · Redis · FCM · Docker Compose

이 프로젝트를 한 줄로 설명하면

분산 채팅 환경에서 메시지 propagation과 조회 consistency를 보강한 프로젝트

PRIMARY ENGINEERING DECISION

메시지 경험은 전송 성공이 아니라 propagation, ordering, read synchronization이 함께 맞아야 신뢰된다.

왜 중요했나

실시간 채팅에서 저장 지연이나 순서 어긋남은 내부적으로는 비동기 처리 지연일 수 있습니다. 하지만 사용자는 이를 메시지가 사라졌거나 대화가 꼬인 것으로 받아들이기 때문에, 전파·저장·조회 시점의 간극을 줄이는 것이 시스템 신뢰성의 핵심이었습니다.

1. 프로젝트 개요

Slack형 협업 메신저를 MSA 구조로 구현한 프로젝트입니다. 채팅, 히스토리, 멤버, 알림 서버의 백엔드 구현과 서버 간 연동을 담당했습니다.

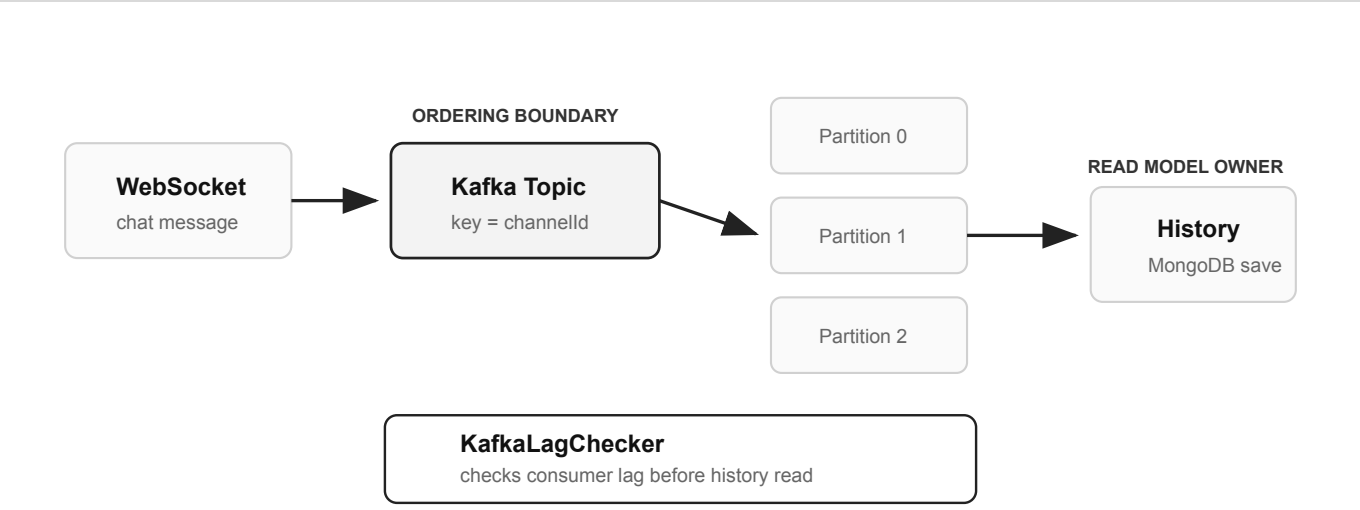
내 역할: WebSocket 채팅 서버, Kafka fan-out, MongoDB History Server, KafkaLagChecker, FCM 알림 서버, 서버 간 JWT 연동.

핵심 문제

사용자는 메시지가 실제로 유실되지 않아도, 화면에서 늦게 보이면 유실로 느낍니다. 그래서 실시간 전파, 저장, 히스토리 조회를 하나의 신뢰 흐름으로 다뤘습니다.

218ms → 34ms 조회 응답 개선 History read path 단순화 기준	200 → 300 동시 접속 확장 팀 부하 테스트 기준	5 × 100ms Lag Polling 최대 5회 offset lag 확인	Channel ID Ordering Boundary Kafka key 기준
---	---	--	--

Measurement note: 응답 시간은 History API의 같은 조회 조건에서 비교했습니다. 개선은 불필요한 조회 경로와 서버 결합을 줄인 결과이며, 운영 SLA가 아니라 구조 변경 전후 확인 지표입니다.



사용자가 체감하는 순서 보장 범위를 채널 단위로 정의하고, Channel ID를 Kafka key로 사용했습니다.

DEEP DIVE

Propagation

Main Narrative. 다중 서버에서 메시지가 같은 경험으로 도착해야 했다

다른 서버에 연결된 사용자가 메시지를 받지 못하면 사용자에게는 장애로 보입니다. 서버 간 직접 호출 대신 Kafka를 이벤트 경계로 두었습니다.

실제로 부딪힌 문제
WebSocket 연결이 특정 서버에 붙어 있기 때문에, 서버가 늘어나는 순간 “내 서버의 구독자”와 “다른 서버의 구독자”가 갈라졌습니다. 직접 호출 방식은 서버 수가 늘수록 전파 경로와 실패 지점이 함께 늘어났습니다.

PROBLEM

WebSocket 연결은 특정 인스턴스에 붙어 있어 서버 확장 시 전파 경로가 끊길 수 있었습니다.

DECISION

Kafka topic을 서버 간 이벤트 경계로 두고 fan-out 책임을 분리했습니다.

CORE IMPLEMENTATION	STOMP handler	Kafka producer/consumer
----------------------------	---------------	-------------------------

결론. 해당 난관 확장 이후에도 메시지 전달 책임이 특화된 구조로 유지됩니다.

결론: 재당 서머 확장 이후에도 메시지 전파 책임이 중립인 구조도 유지했습니다.

DEEP DIVE

Ordering & Synchronization

Deep Dive 1. ordering boundary를 채널 단위로 제한

사용자가 체감하는 순서 보장 범위를 “채널 내부”로 제한하고, Kafka key를 Channel ID로 결정했습니다.

PROBLEM

순서를 보장해야 하는 단위를 먼저 정하지 않으면 Kafka partition 전략이 흔들렸습니다.

DECISION

“같은 채널 안의 대화”를 ordering boundary로 정의했습니다.

CORE IMPLEMENTATION

channelId key

partition ordering

결론: Kafka ordering을 기술 기능이 아니라 사용자 대화 맥락 기준으로 설계했습니다.

Deep Dive 2. 저장 지연을 유실처럼 보이지 않게 만들기

전송 직후 히스토리에 메시지가 없으면 사용자는 “보낸 메시지가 사라졌다”고 느낍니다. DB 조회 전에 consumer lag을 확인해 저장 지연과 실제 유실을 구분했습니다.

디버깅 흐름

메시지 전송은 성공했는데 직후 히스토리 조회에서 누락되는 케이스를 확인했습니다. DB 저장 실패가 아니라 consumer가 아직 처리하지 못한 propagation delay였고, latest offset과 committed offset 차이를 확인하는 방식으로 원인을 분리했습니다.

PROBLEM

실제 유실이 아니어도 저장 지연은 사용자에게 메시지 유실처럼 보였습니다.

DECISION

History API가 Kafka consumer lag을 확인한 뒤 MongoDB를 읽도록 했습니다.

CORE IMPLEMENTATION

KafkaLagChecker

latest offset

committed offset

결론: 비동기 저장 구조를 유지하면서 조회 시점의 정합성을 보강했습니다.

Deep Dive 3. real-time path와 read model 분리

채팅 서버는 실시간 전달에 집중하고, 저장/조회/삭제 정책은 History Server로 분리했습니다.

PROBLEM

실시간 전송과 히스토리 관리는 데이터 모델과 변경 이유가 달랐습니다.

DECISION

Kafka 이벤트를 경계로 서버 책임을 분리했습니다.

CORE IMPLEMENTATION

History Server

MongoDB Document

결론: 채팅 서버 변경 없이 히스토리 조회 정책을 발전시킬 수 있는 구조가 됐습니다.

보조 구현

Profile/JWT: 채팅·히스토리 서버에서 고정 프로필 값을 제거하고, 서버 간 HTTP 호출과 HTTP/STOMP header 양쪽의 JWT 추출 흐름을 정리했습니다.

Notification: 채팅 서버가 알림 책임까지 가지지 않도록 Kafka 이벤트를 소비하는 Notification Server MVP와 FCM Web Push 전송 흐름을 분리했습니다.

Docker Compose: chat-server 단독, history-server 단독, 통합 실행 환경을 분리해 MSA 로컬 실행 재현성을 높였습니다.

OPERATIONAL DECISION

Channel ID key는 채널 내 consistency를 얻는 대신 hot partition 위험을 감수합니다. 운영에서는 partition별 lag, 특정 channel traffic skew, polling timeout 비율을 봐야 합니다. 최대 500ms 이후에도 lag이 남으면 fallback 응답을 주고, replay와 idempotent message key로 중복 저장을 막는 기준이 필요합니다.

실제 구현

Kafka 기반 WebSocket fan-out, Channel ID key ordering, KafkaLagChecker polling, History Server 분리

이후 고려한 방향

hot partition 감시, timeout fallback, consumer failure replay, idempotent message 처리

회고

분산 채팅에서 전송 성공, 저장 완료, 조회 가능 시점은 서로 다른 문제였습니다. 사용자 신뢰는 이 간극을 얼마나 짧고 예측

03. FaceFriend

이미지 분석 커뮤니티 채팅

2024.03 - 2024.06 · Backend · Spring Boot · WebSocket · STOMP · Redis · JPA

이 프로젝트를 한 줄로 설명하면

presence/currentness 기반으로 읽음 정확도와 실패 피드백을 정리한 프로젝트

PRIMARY ENGINEERING DECISION

읽음 정확도는 저장된 메시지가 아니라 현재 presence가 맞을 때 보장된다.

왜 중요했나

WebSocket 채팅의 품질은 메시지 전송 성공만으로 결정되지 않습니다. 상대가 접속 중인지, 같은 방을 보고 있는지, 실패를 어떻게 알려주는지에 따라 읽음 상태와 사용자 신뢰가 달라졌습니다.

1. 프로젝트 개요

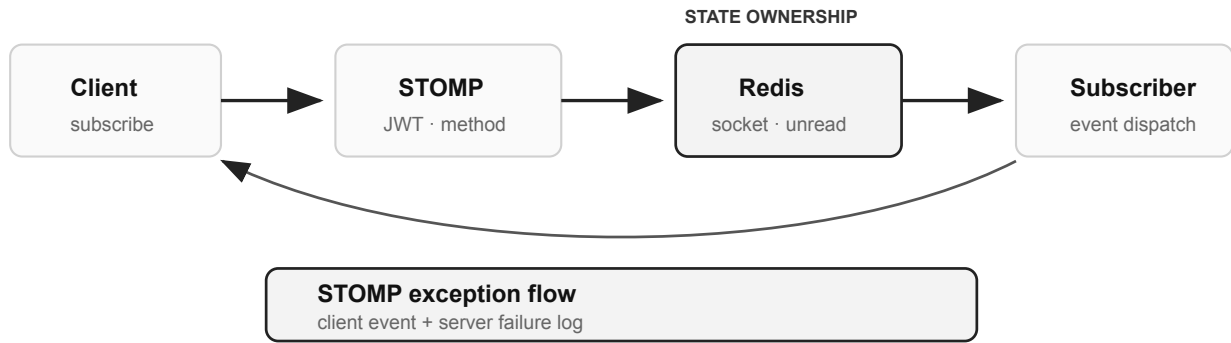
Spring Boot 기반 이미지 분석 커뮤니티에서 WebSocket/STOMP 채팅 기능과 채팅 응답 구조 개선을 담당했습니다.

내 역할: 채팅 도메인, Redis socket/chat 상태 저장소, unread 처리, STOMP 예외 응답, 채팅방 상태별 DTO.

핵심 문제

읽음 상태가 틀리거나 실패가 보이지 않으면 사용자는 채팅이 불안정하다고 느낍니다. 그래서 Redis를 빠른 캐시가 아니라 현재성을 가진 상태 저장소로 사용했습니다.

Presence	Unread Correctness	STOMP Error
socket 접속과 채팅방 입장 상태의 현재성 분리	상대의 current presence에 따라 읽음/미확인 판단	실패도 WebSocket 이벤트로 전달하면서 서버 예외 추적 유지



영속 메시지는 DB에, 접속/입장 상태는 Redis에 두어 실시간 판단과 저장 책임을 분리했습니다.

DEEP DIVE

Currentness & Errors

Main Narrative. presence가 틀리면 unread도 틀린다

읽음 처리는 상대가 지금 접속해 있고 같은 방을 보고 있는지에 따라 달라집니다. 접속 여부와 채팅방 입장 여부를 Redis의 현재 상태로 분리했습니다.

실제로 부딪힌 문제

브라우저 종료나 네트워크 단절처럼 정상 disconnect가 오지 않는 상황에서는 접속 상태가 남을 수 있습니다. 이 stale presence가 남으면 상대가 방에 있는 것처럼 판단되어 unread가 잘못 계산될 수 있었습니다.

PROBLEM

접속 상태는 자주 바뀌며, 오래 보관할 데이터보다 지금 맞아야 하는 데이터였습니다.

DECISION

Redis를 캐시가 아니라 현재성을 가진 runtime state store로 사용했습니다.

CORE IMPLEMENTATION

SocketInfoRedisRepository

ChatRoomInfoRedisRepository

unread flow

결론: 메시지 저장 책임과 실시간 상태 판단 책임을 분리했습니다.

Deep Dive 1. 실패도 실시간 이벤트로 전달

메시지 전송 실패가 화면에 보이지 않으면 사용자는 입력이 무시됐다고 느낍니다. 실패도 실시간 이벤트로 전달하고, 서버에는 추적 가능한 로그를 남겼습니다.

구현 중 어려움

HTTP 예외처럼 응답을 내려줄 수 없어서, 실패도 STOMP destination으로 보내야 했습니다. 동시에 서버 로그에는 원인을 남겨야 했기 때문에 사용자 이벤트와 운영 추적 로그를 분리했습니다.

PROBLEM

실시간 메시징에서는 성공뿐 아니라 실패도 같은 흐름 안에서 다뤄야 했습니다.

DECISION

사용자에게는 STOMP 오류 이벤트를 보내고, 서버에는 실패 흐름을 남겼습니다.

CORE IMPLEMENTATION

STOMP error response

exception tracking

결론: 프론트엔드 사용자 경험과 서버 디버깅 가능성을 동시에 유지했습니다.

OPERATIONAL DECISION

Recovery Criteria

OPERATIONAL DECISION

presence는 빠른 조회보다 stale 상태 제거가 더 중요합니다. heartbeat 누락, connection TTL 만료, reconnect 발생률을 기준으로 cleanup을 판단해야 합니다. Redis 장애나 서버 재시작 이후에는 DB 메시지 기준으로 unread를 재계산하는 fallback 경로가 필요합니다.

실제 구현

Redis socket/chat 상태 저장소, unread 판단 흐름, STOMP 예외 응답, method 기반 이벤트 표준화

이후 고려한 방향

heartbeat, connection TTL, stale state cleanup, reconnect recovery, unread 재계산 정책

회고

presence는 저장된 사실보다 현재성이 중요했습니다. 잘못 남은 연결 상태 하나가 unread 판단 전체를 흔들 수 있다는 점이 핵심이었습니다.

04. 기타 프로젝트 및 활동

핵심 프로젝트 밖에서 이어간 구현, 학습, 협업 경험

OneHabit · KiKi Kiosk · Wakeup · Relanz · Java GC Puzzle · Collaboration

왜 중요했나

작은 프로젝트와 활동에서도 같은 관점이 반복됐습니다. 구현 전에 상태 기준을 정하고, 객체 책임을 나누고, 팀이 같은 판단 근거를 보도록 문서화한 경험은 핵심 프로젝트의 설계 판단을 뒷받침합니다.

HACKATHON

SCHEDULER

OneHabit · 제한 시간 안에서 범위 재정의

하루 1회 인증·연속 인증·마감 기준을 Scheduler로 분리하고, 팀이 같은 상태 기준을 보도록 문서화했습니다.

DOMAIN

OOP

KiKi Kiosk · Use Case에서 객체 책임 도출

Use Case와 Sequence Diagram으로 객체 간 메시지를 먼저 정리하고, GRASP 관점에서 책임을 배치했습니다.

SERVICE

DJANGO

Wakeup / Relanz · 데이터 변경 흐름 정리

Wakeup에서는 알람 확인 이벤트가 EXP/포인트/통계로 전파되는 순서를 잡았고, Relanz에서는 Session 의존 조회를 명시적 View 조회로 바꿨습니다.

TEAM

QUALITY GATE

Collaboration / Java GC Puzzle · 팀 기준과 학습 도구화

ADR 성격의 결정 기록, 리뷰 규칙, CI 검증을 제안했습니다. Java GC는 Heap/Stack/참조 관계를 퍼즐 규칙으로 바꿔 런타임 이해를 도구화했습니다.